

Entity Resolution System (ERSys)

Baseline Architecture Document

Status: Draft – Normative Baseline

Version: 2.0

Date: 28 Jan 2026

0.2 Referenced artefacts and precedence

This architecture document is part of a controlled set of artefacts that collectively define the intent, scope, semantics, and architectural baseline of the Entity Resolution System (ERSys). Each artefact has a distinct role and authority. Precedence is defined by concern rather than by a single linear hierarchy.

The following artefacts are referenced:

- Product Requirements Document (PRD)
- Implementation Offer
- White and Blue Use Case Catalogue
- Architecture Decision Records (ADRs)
- This Architecture Document
- ERS–ERE Technical Contract and ERE Developer Guide

Authority and precedence by concern

- The Product Requirements Document (PRD) and the Implementation Offer are authoritative with respect to overall intent, high-level scope, and commercial commitments. They define what is to be delivered and what is explicitly out of scope, but they do not define detailed system semantics or behavioural guarantees.

- White Use Cases are authoritative with respect to externally observable behaviour and system guarantees. They define what ERSys must do, under which conditions, and with which observable outcomes, without prescribing internal implementation.
- Architecture Decision Records (ADRs) are authoritative with respect to binding architectural decisions, invariants, and semantic choices that constrain the system design and evolution.
- This Architecture Document consolidates and structures the intent, guarantees, and decisions defined in the artefacts above into a coherent architectural baseline. It is authoritative with respect to system boundaries, responsibility allocation, structural organisation, and the consistent interpretation of services, interfaces, and components.
- Blue Use Cases refine internal behaviour and implementation considerations. They are subordinate to White Use Cases, ADRs, and this Architecture Document.
- Implementation artefacts, including code, configuration, and deployment descriptions, are subordinate to all of the above.

Conflict resolution

In case of ambiguity or conflict:

- vague or high-level statements in the PRD or Implementation Offer shall not override explicit guarantees, invariants, or decisions defined in White Use Cases, ADRs, or this Architecture Document;
- inconsistencies between White Use Cases, ADRs, and this Architecture Document shall be resolved in favour of the more specific and explicitly normative statement;
- any genuine conflict affecting contractual scope or intent shall be resolved through formal change to the PRD and corresponding update of the architectural baseline.

This precedence model ensures that ambiguity does not override precision and that architectural authority remains explicit, stable, and reviewable throughout the lifecycle of ERSys.

Table of contents

Entity Resolution System (ERSys)	1
Baseline Architecture Document	1
0.2 Referenced artefacts and precedence	1
Table of contents	3
1. Introduction and reading guide	4
1.1 Purpose of this document	4
1.2 Intended audience	4
1.3 How to read this document	5
1.4 Terminology and interpretation	5
1.5 Diagram notation and interpretation	5
2. Architectural scope and boundary conditions	6
2.1 System-of-interest	6
2.2 Responsibilities within the architectural boundary	7
2.3 Outside the architectural boundary	7
2.4 Explicit non-goals of this architectural baseline	8
2.5 Interpretation and evolution	9
3. Actors, roles, and responsibility boundaries	9
3.1 Actors and roles	10
3.2 Ownership and responsibility boundaries	12
3.3 Interpretation and change control	12
4. Architectural decision baseline	12
4.1 Architectural principles	12
4.2 Normative architectural invariants	13
4.3 Architecture Decision Records (ADR) index	15
Foundational ADRs	15
Structural ADRs	15
Policy ADRs	16
4.4 Interpretation and change control	17
5. Business context and quality attributes	17
5.1 Business context and drivers	17
5.2 Quality attributes and architectural priorities	18
6. System context and cooperation	19
6.1 System context and boundary (Business L1)	19
6.2 Application cooperation and responsibility allocation (Application L2)	20
6.3 Interfaces and contracts	22
6.4 Interpretation	23
7. Core system capabilities (White Use Cases)	23
7.1 Resolve Entity Mention (UC-W1)	24
7.2 Curate Resolution Decisions (UC-W2)	26
7.3 Rebuild Resolution State (UC-W3)	28
Main Success Scenario	30

UC-W4: Consult Resolution Statistics	31
7.5 Relationship to detailed behaviour and architectural decisions	33

1. Introduction and reading guide

1.1 Purpose of this document

This document defines the normative system architecture for the Entity Resolution System (ERSys). It translates the agreed Product Requirements Document (PRD) into a coherent and implementable architectural baseline.

The purpose of this document is to establish architectural authority. It specifies system boundaries, responsibilities, behavioural guarantees, and architectural invariants that must be respected by all components of ERSys and by any implementation claiming conformance to this architecture.

This document is not a feature catalogue, a detailed design specification, or an operational manual. It deliberately avoids prescribing internal algorithms, technology stacks, deployment tooling, or user-interface implementations. Instead, it defines the minimum set of architectural commitments required to enable consistent and interoperable implementations, support safe integration with external systems, preserve semantic integrity over time, and allow controlled evolution without architectural drift.

Where multiple design or implementation options exist, only those decisions that are architecturally significant for system coherence and integrity are captured here. Detailed artefacts such as API specifications, schemas, configuration files, deployment descriptors, and operational procedures are referenced where necessary but are maintained outside this document.

In case of conflict, this architecture document takes precedence over earlier solution descriptions, prototype documentation, or informal design notes, unless explicitly superseded by a later version of this document.

1.2 Intended audience

This document is intended for the following audiences:

- Product owners and commissioning stakeholders, who require a stable and reviewable description of system scope, guarantees, and explicit non-goals.
- Enterprise, solution, and system architects, responsible for positioning ERSys within a broader architectural landscape and ensuring boundary alignment.
- Implementation and integration roles, who require clear responsibility separation, explicit contracts, and well-defined behavioural expectations.
- Reviewers and auditors, who need to understand what ERSys is accountable for, what it explicitly does not do, and how architectural consistency is enforced.

Readers are not expected to be familiar with internal resolution algorithms or specific technology choices. They are expected to be comfortable with architectural concepts such as system boundaries, contracts, authority, and invariants.

1.3 How to read this document

This document is structured to progress from authority and intent, through behavioural guarantees, to structural realisation.

Guidance for readers:

- Sections 2–4 establish architectural authority by defining system boundaries, roles, responsibility separation, architectural principles, and non-negotiable invariants.
- White-level Use Cases define the externally observable behaviour of ERSys. They specify what the system guarantees under defined conditions, without constraining internal algorithms or technologies.
- Behavioural spine flows identify the small set of interaction patterns that fundamentally characterise the system: resolution intake, asynchronous engine interaction, human curation, and rebuild operations. These flows are architectural commitments rather than illustrative examples.
- Architecture Decision Records (ADRs) capture the rationale behind non-trivial architectural choices. ADRs provide historical and explanatory context; their normative force is subordinate to the PRD and to the architecture text itself.
- Annexes contain supporting material such as Blue-level Use Cases, the Business Glossary, ADR texts, and contract references. These annexes support consistent interpretation but do not replace the normative statements in the main body.

Readers seeking a rapid understanding of ERSys authority and guarantees should focus on Sections 2, 4, 7, and 8. Readers involved in implementation, integration, or review should treat the document as a whole.

1.4 Terminology and interpretation

The Business Glossary (Annex A) defines the authoritative meaning of domain and system terms used throughout this document.

Unless explicitly stated otherwise, all occurrences of glossary terms, whether capitalised or not, are interpreted according to their glossary definitions and are used consistently across architectural views, use cases, decision records, and interface descriptions.

Where a term is used in a specialised or contextual sense that deviates from its glossary definition, that deviation is stated explicitly and applies only within the stated scope.

This rule is normative. It ensures consistent interpretation across the architecture, prevents semantic drift as the system evolves, and supports traceability between requirements, architectural decisions, and implementation artefacts.

1.5 Diagram notation and interpretation

This document uses a small, controlled set of modelling notations to express architectural structure, behaviour, and information concepts.

ArchiMate notation is used as the primary enterprise architecture language, particularly for Business and Application layer views that describe system boundaries, actors, roles, application components, and their relationships. These diagrams are normative with respect to scope, responsibility, and interaction boundaries.

UML sequence diagrams are used selectively to illustrate behavioural spine flows. They express interaction order and responsibility transfer but do not introduce additional structural elements beyond those defined in ArchiMate views.

UML class diagrams are used for conceptual information modelling, including domain concepts and data structures relevant to entity resolution. These models describe conceptual structure and relationships and do not prescribe physical schemas, storage technologies, or persistence strategies.

In some cases, diagrams adopt composition and containment conventions inspired by the C4 model to improve readability. Where this occurs, C4-style composition is applied using ArchiMate elements and semantics and does not introduce additional modelling concepts or abstraction levels.

All diagrams are to be interpreted according to the semantics of their underlying notation. Visual layout, grouping, or proximity alone must not be interpreted as implying behaviour, ownership, or dependency unless explicitly represented by a defined relationship.

2. Architectural scope and boundary conditions

This section defines the authoritative architectural scope of the Entity Resolution System (ERSys). It establishes what is inside the system boundary, which responsibilities are owned by ERSys, and which concerns are explicitly excluded from this architectural baseline.

The scope, responsibilities, exclusions, and non-goals defined here are normative. All use cases, architectural views, interaction flows, contracts, and decision records in this document shall be interpreted and evaluated against the boundary conditions stated in this section.

2.1 System-of-interest

The system-of-interest is the Entity Resolution System (ERSys).

ERSys is a distributed system whose sole architectural responsibility is to assign, expose, and govern canonical identifiers for entity mentions submitted by external systems.

ERSys provides an identity governance layer that supports downstream interoperability and traceability. It does not assert factual truth about real-world entities and does not materialise enriched, consolidated, or authoritative entity data.

All system parts that participate in canonical identity assignment, resolution decision governance, or canonical state exposure are considered internal to the ERSys boundary for

architectural purposes, regardless of their concrete implementation, deployment topology, or operational ownership.

2.2 Responsibilities within the architectural boundary

The following responsibilities are within the architectural boundary of ERSys and are mandatory for any conformant implementation.

ERSys is responsible for:

- Canonical identity assignment. Assigning a canonical identifier to every valid entity mention submitted to the system within bounded time, treating the identifier strictly as an identity reference and not as a materialised entity record.
- Canonical state exposure. Exposing canonical identifiers and the current mention-to-canonical assignment state to external consumers through stable, contractually defined interfaces.
- Resolution orchestration. Coordinating automated resolution performed by one or more internal resolution capabilities, including asynchronous request forwarding, response correlation, and bounded integration of results.
- Decision-centric governance. Managing resolution outcomes through explicit resolution decisions, including decision status, candidate context, and curator-imposed constraints.
- Human-in-the-loop curation. Supporting governed human intervention that allows explicit acceptance, rejection, or reassignment of canonical links in accordance with defined use cases.
- Rebuild and corrective processing. Supporting controlled system-level rebuild operations that recompute resolution state while preserving authoritative governance constraints.
- Persistence of authoritative records. Persisting all submitted entity mentions and all resolution decisions as authoritative records sufficient to enable replay, rebuild, and traceability of system behaviour.

These responsibilities are realised and constrained by the behavioural guarantees defined in the White-level Use Cases.

2.3 Outside the architectural boundary

The following concerns are explicitly outside the architectural boundary of ERSys. They may exist in the surrounding ecosystem but are not governed, implemented, or constrained by this architecture.

ERSys does not include responsibility for:

- Resolution engine internals. Internal algorithms, data structures, optimisation strategies, or persistence mechanisms of resolution engines, provided that external contracts are respected.

- User interface internals. Presentation logic, user experience design, and internal state management of user-facing applications, beyond their interaction with ERSys through published APIs.
- Upstream ingestion pipelines. Systems that produce, transform, or pre-process entity mentions prior to submission, beyond compliance with ERSys input contracts.
- Downstream consumer behaviour. Semantics, persistence strategies, or usage patterns adopted by systems consuming canonical identifiers.
- Authentication and identity infrastructure. Authentication, identity management, and single sign-on mechanisms, beyond role-level assumptions required by use cases.

2.4 Explicit non-goals of this architectural baseline

The following items are absolute non-goals of the current ERSys architectural baseline. Their exclusion is deliberate and foundational. They shall not be inferred implicitly from other sections of this document.

Representation and data semantics

ERSys does not materialise canonical entities as enriched, consolidated, or authoritative data records.

ERSys does not perform attribute merging, golden-record construction, or master data management.

ERSys does not assert factual truth about real-world entities. It manages identity hypotheses under available evidence.

ERSys does not apply domain-specific semantic models or ontologies beyond minimal schema definitions required for parsing and validation.

Data processing and enrichment

ERSys does not clean, normalise, enrich, or repair incoming entity data.

ERSys does not reconcile entity mentions against external authoritative datasets.

ERSys does not perform cross-source data harmonisation.

Learning, analytics, and feedback loops

ERSys does not retrain resolution models or embeddings based on curator actions.

ERSys does not manage machine learning lifecycles, experiments, or performance tuning.

ERSys does not provide analytical dashboards or statistical insights beyond lightweight operational signals required for observability.

Governance, audit, and access control

ERSys does not provide formal or compliance-grade audit guarantees.

ERSys does not define or mandate specific authentication or access-control technologies.

ERSys does not support fine-grained or attribute-level access control beyond role-level separation required by use cases.

2.5 Interpretation and evolution

The scope, responsibilities, exclusions, and non-goals defined in this section constrain the interpretation of all use cases, diagrams, contracts, and decision records in this document.

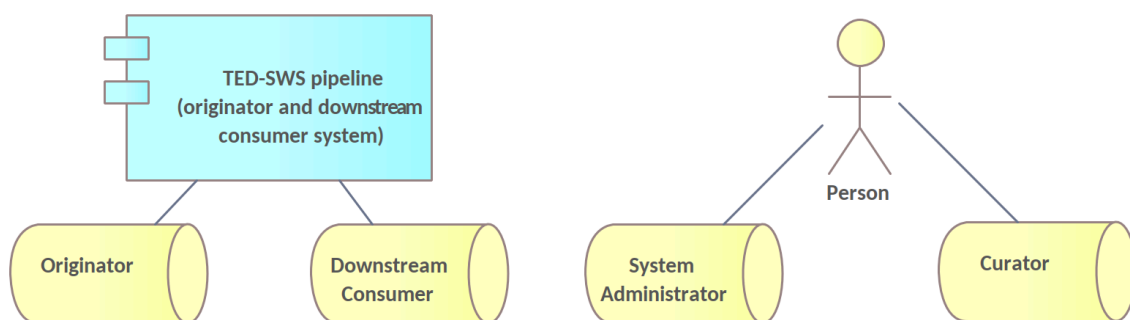
Any extension or modification that violates these boundary conditions requires explicit revision of this section and corresponding updates to the architecture document. Such changes constitute architectural evolution and shall not be introduced implicitly through implementation decisions, configuration changes, or undocumented operational practices.

This boundary discipline is mandatory to preserve predictability, governance integrity, and long-term evolvability of ERSys.

3. Actors, roles, and responsibility boundaries

This section defines the actors and roles that interact with the Entity Resolution System (ERSys) and establishes the authority boundaries that govern those interactions. Roles are defined independently of organisational structure, deployment topology, or delivery responsibilities. All interactions with ERSys are evaluated strictly according to the role assumed in a given interaction.

The diagram below provides a conceptual overview of the external actors that interact with ERSys and the architectural roles they assume when doing so.



A single external system or person may assume multiple roles over time. For example, the same external system may act both as an Originator when submitting entity mentions and as a Downstream Consumer when consuming canonical identifiers.

Roles shown in this diagram define interaction authority, not organisational structure. All permissions, constraints, and responsibilities are governed exclusively by the role semantics defined in this section, regardless of deployment topology or implementation detail.

3.1 Actors and roles

ERSys recognises a limited and closed set of roles. An actor may assume one or more roles over time, but each interaction with the system is governed exclusively by the authority and constraints of the role assumed for that interaction.

API Client (Originator)

Role description: submits entity mentions to ERSys for resolution and receives canonical identifiers.

Permitted actions:

- Submit entity mentions for resolution through published resolution APIs.
- Receive canonical identifiers and correlated resolution outcome statuses.

Authority constraints:

- Does not influence resolution logic, matching strategies, or governance rules.
- Does not modify resolution decisions or mention-to-canonical assignments.
- Does not issue canonical identifiers.

Curator

Role description: exercises human governance over resolution outcomes.

Permitted actions:

- Review automated resolution outcomes and candidate alignments.
- Accept, reject, or reassign canonical links through governed curation actions.

Authority constraints:

- Overrides automated outcomes only through explicit resolution decisions.
- Does not perform resolution computation.
- Does not manage system configuration, deployment, or operational behaviour.

System Administrator

Role description: performs controlled system-level administrative operations.

Permitted actions:

- Initiate rebuild and maintenance operations as defined by administrative use cases.
- Manage curator access and active sessions at role level.

Authority constraints:

- Does not create, modify, or override individual resolution decisions.
- Does not influence automated resolution logic or candidate evaluation.

Operator / Developer

Role description: observes and operates ERSys in a technical capacity.

Permitted actions:

- Observe system health, performance, and failure conditions through operational interfaces.
- Perform technical diagnostics and recovery actions that do not alter domain behaviour.

Authority constraints:

- Does not submit entity mentions.
- Does not influence resolution outcomes, curation decisions, or canonical state.

Entity Resolution Engine (Supporting System)

Role description: provides computational resolution capabilities to ERSys.

Permitted actions:

- Receive resolution requests from ERSys.
- Execute resolution computation and return candidate alignments with associated assessment signals.

Authority constraints:

- Proposes candidate outcomes only.
- Does not issue canonical identifiers.
- Does not persist authoritative records or governance decisions.

Downstream Consumer

Role description: consumes canonical identifiers produced by ERSys as part of downstream processing.

Permitted actions:

- Retrieve canonical identifiers and current assignment state through supported lookup or projection mechanisms.

Authority constraints:

- Does not submit entity mentions.
- Does not modify resolution outcomes or governance decisions.
- Interacts with ERSys exclusively through read-only interfaces.

3.2 Ownership and responsibility boundaries

ERSys enforces strict ownership boundaries between its internal responsibilities and supporting systems. These boundaries are architectural invariants and apply regardless of organisational or deployment arrangements.

- The Entity Resolution Service (ERS) owns architectural authority. It is responsible for orchestration, enforcement of governance rules, persistence of authoritative records, and exposure of all external application services.
- The Entity Resolution Engine (ERE) owns computation. It executes resolution logic and proposes candidate alignments under the ERS–ERE contract without owning or mutating authoritative state.
- The Link Curation application owns user-facing workflows. It supports curator and administrator interactions and accesses ERSys exclusively through ERS-managed APIs, without direct access to internal state or resolution engines.

These boundaries ensure that governance dominates automation, authoritative state is managed in a single place, and computational strategies remain replaceable without affecting external semantics.

3.3 Interpretation and change control

The roles and authority boundaries defined in this section constrain the interpretation of all subsequent use cases, interaction flows, and architectural views.

Any extension that introduces new roles, expands role authority, or weakens these boundaries constitutes an architectural change. Such changes require explicit revision of this section and corresponding updates to the system architecture baseline. They shall not be introduced implicitly through implementation choices, configuration, or undocumented operational practice.

4. Architectural decision baseline

This section defines the normative architectural baseline of the Entity Resolution System (ERSys). It states the binding architectural principles and invariants that constrain all implementations and govern system behaviour independently of technology choices, deployment models, or internal algorithms.

All use cases, architectural views, interaction flows, contracts, and decision records in this document shall be interpreted as applications of the baseline defined in this section.

4.1 Architectural principles

The following architectural principles are binding. They express non-negotiable rules that shape all design and implementation choices within ERSys.

Authority of ERS over identity

The Entity Resolution Service (ERS) is the sole architectural authority responsible for issuing canonical identifiers and maintaining the current mention-to-canonical assignment state.

No other component, system, or actor may assert, modify, or invalidate canonical identity assignments independently of ERS.

Asynchronous, best-effort computation

Automated resolution is performed asynchronously by resolution engines and integrated on a best-effort basis.

Client-facing behaviour of ERSys shall not be coupled to the availability, performance, or completion of resolution engines. Resolution engines contribute proposals, not guarantees.

Manual curation dominates automation

Explicit curator actions are authoritative and always take precedence over automated resolution outcomes.

Automated resolution results are treated exclusively as proposals that may be accepted, rejected, or overridden through governed curation actions.

4.2 Normative architectural invariants

The following invariants define the architectural baseline of ERSys. They are mandatory and apply across all system components, interaction flows, and conformant implementations.

Canonical identifier stability

Canonical identifiers, once issued, shall remain stable and shall never be reassigned or invalidated.

Changes over time affect mention-to-canonical assignments, not the identifiers themselves.

One current decision per mention and entity type

For each entity mention and entity type, there shall be at most one current resolution decision governing its canonical assignment.

Superseded or historical decisions shall not affect the current canonical projection.

For each entity mention and entity type, there shall be at most one current resolution decision governing its canonical assignment.

Superseded or historical decisions shall not affect the current canonical projection.

Curator decisions override automated outcomes

Manual curation actions, including explicit acceptance or rejection of alignments, shall always override automated resolution results.

Automated outcomes shall not bypass, negate, or overwrite curator-imposed constraints.

Bounded resolution guarantee

Every valid entity mention submitted to ERSys shall yield a canonical identifier within a bounded time.

This guarantee applies regardless of the completion state of asynchronous resolution processing.

Every valid entity mention submitted to ERSys shall yield a canonical identifier within a bounded time.

This guarantee applies regardless of the completion state of asynchronous resolution processing.

Engine failure isolation

Failures, delays, or timeouts in resolution engines shall not block, delay, or invalidate client-facing responses produced by ERSys.

Separation of authoritative and operational state

Authoritative resolution state, including resolution decisions and canonical projections, shall be persisted and governed exclusively by ERSys.

Operational or intermediate state maintained by resolution engines is non-authoritative and shall not be relied upon for governance or exposure.

Authoritative resolution state, including resolution decisions and canonical projections, shall be persisted and governed exclusively by ERSys.

Operational or intermediate state maintained by resolution engines is non-authoritative and shall not be relied upon for governance or exposure.

Deterministic replay and rebuild

The authoritative system of records maintained by ERSys shall be sufficient to deterministically replay resolution processing and to reconstruct canonical state during rebuild operations.

Curator-imposed constraints shall be preserved across all rebuilds.

All invariants in this section are enforced through the behaviour specified in the White-level Use Cases and are further explained, where applicable, by the Architecture Decision Records.

4.3 Architecture Decision Records (ADR) index

Architecture Decision Records (ADRs) document the rationale behind non-trivial architectural choices that underpin the ERSys baseline. ADRs complement the normative principles and invariants defined in this section but do not override them.

ADRs are grouped by stability class to support architectural review, onboarding, and controlled evolution. Stability classes indicate the expected impact and governance required for change.

Foundational ADRs

Foundational ADRs define client-facing semantics, authority boundaries, and non-negotiable invariants. Changes to these ADRs are exceptional and constitute breaking architectural change. These ADRs underpin the identity, authority, and bounded-response invariants defined in Section 4.2.

Domain A: Identity semantics and client guarantees

- ADR-A1: Canonical Identifier Derivation and Governance by Construction. Defines canonical identifiers as opaque, deterministically derived identity anchors based on (`sourceId`, `sourceRequestId`, `entityType`).
- ADR-A2: Non-Empty Resolution Guarantee. Establishes that every valid resolution request yields a canonical identifier, including explicit singleton identities for no-match cases.
- ADR-A3: Identity Monotonicity. Establishes that canonical identifiers are never revoked, even as mention-to-canonical assignments evolve.

Domain B: Authority and governance model

- ADR-B1: ERS–ERE Responsibility Split. Establishes ERS as authoritative for identity, decisions, and governance, with ERE operating in a non-authoritative advisory role.
- ADR-B2: Decision-Centric Governance Model. Defines resolution decisions as the sole unit of governance, with exactly one active decision per request.
- ADR-B3: Curator Authority and Lock Semantics. Establishes curator confirmations and rejections as immutable locks that override automated outcomes.
- ADR-B4: Normative Confidence Semantics. Defines confidence as a discrete governance signal (`+1`, `0`, `-1`), distinct from engine-specific similarity scores.

Structural ADRs

Structural ADRs shape system interaction, state management, and operational correctness. Changes require careful migration planning and explicit impact assessment.

Domain C: Distributed interaction and failure semantics

- ADR-C1: Asynchronous REST and Eventual Consistency Client Interaction. Defines immediate REST responses with asynchronous processing and convergence through read-only lookup.
- ADR-C2: ERS–ERE Messaging and Delivery Guarantees. Establishes asynchronous, at-least-once messaging with idempotency, no ordering assumptions, and request-level correlation.
- ADR-C3: Late Response, Duplicate Response, and Error Absorption Rules. Normalises lateness, duplication, and engine errors while preserving governance invariants.

Domain D: State authority and persistence model

- ADR-D1: State Separation and Authoritative Stores. Separates immutable intake, governance state, and canonical serving state with explicit authority boundaries.

Domain E: Corrective operations and consistency preservation

- ADR-E1: Rebuild Semantics and Governance Preservation. Defines rebuild as an explicit, scoped, administrator-triggered corrective operation that preserves locks and identity invariants.
- ADR-E2: Re-Resolution with Rejections Protocol. Separates read-only lookup from explicit re-resolution under rejection constraints.

Policy ADRs

Policy ADRs capture deliberately isolated and revisitable choices. Changes to these ADRs do not invalidate foundational invariants but may affect behaviour or integration contracts.

Domain F: Representation and payload boundaries

- ADR-F1: Representation Symmetry at the ERS–ERE Boundary. Requires ERS and ERE to operate on the same domain-driven, engine-agnostic representations.
- ADR-F2: No Canonical Representation Materialisation by ERS. Restricts ERS to identity anchoring and forbids authoritative canonical payloads.
- ADR-F3: UI Preview Extraction is Non-Authoritative. Isolates UI preview data as best-effort, presentation-only information.
- ADR-F4: Candidate Handling and Top-N Pruning Policy. Centralises candidate pruning and threshold usage within ERS and forbids boundary-level filtering.

Domain G: Versioning and lineage policy

- ADR-G1: No Exposed Version History or Lineage Graph for Canonical Entities. Restricts clients to the latest canonical state and forbids exposed lineage or version history.

Domain H: Observability and telemetry

- ADR-H1: Observability and Telemetry Semantics. Defines best-effort, metadata-only observability that does not affect system behaviour or contracts.

Domain I: Authentication and authorisation

- ADR-I1: Authentication via External SSO; ERS Retains Authorisation Only.

Detailed ADR texts are provided in Annex C.

4.4 Interpretation and change control

The architectural principles and invariants defined in this section constitute the current normative baseline for ERSys.

Any change that weakens, bypasses, or violates these rules constitutes an architectural change and requires explicit revision of this section and corresponding updates to the architecture document.

Such changes shall not be introduced implicitly through implementation decisions, configuration choices, or undocumented operational practices.

5. Business context and quality attributes

This section provides the business and quality context within which the Entity Resolution System (ERSys) operates. It explains the environment and pressures that informed the architectural baseline defined in Sections 2–4.

The content of this section is informative. It introduces no new requirements, constraints, or guarantees and shall not be used to reinterpret or override the normative architectural rules defined elsewhere in this document. Where a tension exists between business context and architectural invariants, the invariants prevail.

5.1 Business context and drivers

ERSys operates in an environment where multiple independent systems need to exchange and interpret references to real-world entities without sharing ownership of underlying data, aligning on complete data models, or assuming uniform data quality.

The primary business drivers that shaped the ERSys architecture are:

Interoperability across systems

Independent systems require a shared and stable way to refer to entities while remaining free to manage their own data representations, semantics, and quality processes. ERSys addresses this need by providing a canonical identity layer that enables interoperability without imposing domain-specific models or centralised data ownership.

Governance of resolution outcomes

In domains where entity resolution has legal, financial, or operational consequences, automated matching alone is insufficient. Resolution outcomes must remain subject to explicit human governance, allowing context-aware correction and accountability when automated results are inadequate or ambiguous.

Operational scale and throughput pressure

ERSys is deployed in contexts characterised by high-volume, pipeline-driven data flows. Architectural decisions therefore prioritise decoupling, elasticity, and incremental scalability in order to accommodate variable workloads without introducing tight coupling between external clients and internal processing components.

Evolution without lock-in

Resolution strategies, engines, and heuristics evolve over time. The system must accommodate such evolution while preserving identifier continuity and without forcing downstream consumers to adapt to internal changes.

5.2 Quality attributes and architectural priorities

The following quality attributes represent the priorities that informed architectural trade-offs during the design of ERSys. They explain why certain architectural choices were preferred over alternatives; they do not constitute independent requirements.

Predictable and bounded behaviour

In time-sensitive integration scenarios, predictability is valued over exhaustive resolution. The architecture therefore favours bounded and deterministic system behaviour over completeness or optimisation of automated resolution.

Correctness under manual intervention

When automated outcomes conflict with domain understanding or contextual knowledge, explicit curator actions are treated as authoritative. Architectural choices therefore prioritise correctness under human governance over maximising automated accuracy.

Stability of identifiers

Long-lived identifier stability is essential for referential integrity across systems. Architectural options that would jeopardise identifier continuity were deprioritised in favour of approaches that preserve stability over time.

Replaceability of resolution engines

Resolution engines are treated as interchangeable computational components. The architecture avoids coupling authoritative state or external behaviour to engine-specific assumptions in order to preserve replaceability.

Resilience to partial failure

The system is expected to operate in environments where partial failure is normal. Architectural choices therefore favour fault isolation and graceful degradation over end-to-end synchronisation.

Traceability over auditability

ERSys emphasises traceability sufficient to understand and explain resolution outcomes, without aiming to satisfy formal audit or regulatory compliance regimes. This reflects a deliberate balance between transparency and operational complexity.

6. System context and cooperation

This section defines the structural context of the Entity Resolution System (ERSys) and the cooperation patterns between ERSys and its surrounding systems. It establishes clear system boundaries, externally visible services, internal application responsibilities, and the contracts through which cooperation is realised.

The content of this section is structural and descriptive. It introduces no behavioural sequencing, timing semantics, or lifecycle rules. All execution order, state transitions, and behavioural guarantees are specified exclusively through use cases and interaction diagrams in later sections.

The architectural views in this section are expressed using ArchiMate notation. They may be read as analogous to C4 Context and Container views, but ArchiMate is the sole normative modelling language for interpretation.

6.1 System context and boundary (Business L1)

At the business layer, ERSys is a coherent system that exposes a small, stable set of externally visible services to clearly defined roles. These services constitute the contractual boundary of ERSys and define the only supported modes of interaction with external actors and systems.

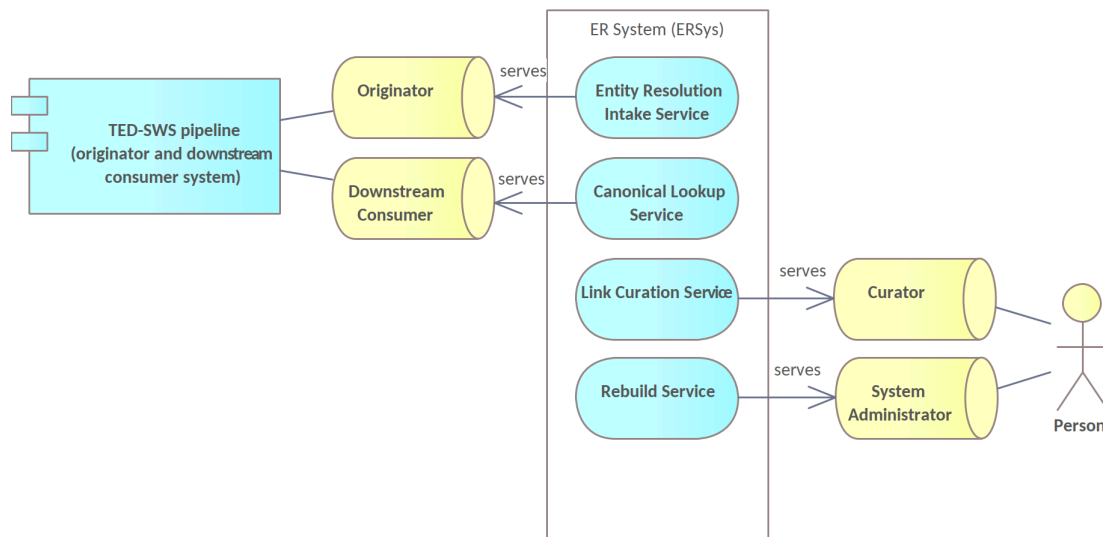
ERSys provides the following business services:

- **Entity Resolution Intake Service**, serving the Originator role for submission of entity mentions and issuance of canonical identifiers.
- **Canonical Lookup Service**, serving the Downstream Consumer role for retrieval of canonical identifiers and current mention-to-canonical assignments.
- **Link Curation Service**, serving the Curator role for governed review and correction of resolution outcomes.
- **Rebuild Service**, serving the System Administrator role for controlled system-level corrective processing.

External systems interact with ERSys exclusively through these services. An external system may assume one or more roles concurrently. For example, the TED-SWS pipeline

assumes both the Originator and Downstream Consumer roles. This example is illustrative and does not constrain the architecture to a specific integration.

The Business L1 view makes explicit that ERSys is responsible for canonical identity assignment and governance, while external systems remain responsible for their own data semantics, processing logic, and downstream usage.



This diagram presents the **business-level system context** of ERSys, showing which roles interact with the system and which externally visible services form its contractual boundary. The diagram uses **ArchiMate Business and Application layer concepts**. Business roles represent interaction authority, while application services represent externally exposed capabilities. The ERSys boundary indicates architectural responsibility, not deployment or ownership.

External systems, such as the TED-SWS pipeline, are shown only to illustrate role assumption patterns. The diagram does not imply coupling, sequencing, or technology choices. It makes explicit that ERSys governs canonical identity and resolution decisions, while external systems retain responsibility for their own data semantics and processing.

Readers should interpret this view strictly as a boundary and responsibility map, not as an execution or integration flow.

6.2 Application cooperation and responsibility allocation (Application L2)

At the application layer, ERSys is realised as a small set of cooperating application components with clearly separated responsibilities and explicit authority boundaries.

Entity Resolution Service (ERS)

ERS is the authoritative application component within ERSys. It:

- exposes all external application interfaces corresponding to the business services;
- orchestrates resolution and curation processes at application level;
- enforces architectural invariants and governance rules;
- persists all authoritative records.

ERS is the sole application component that issues canonical identifiers and manages resolution decisions.

Link Curation Application

The Link Curation Application is a user-facing application that supports curator and administrator interactions. It is outside the ERSys system-of-interest. The application accesses ERSys exclusively through the Link Curation REST API and has no direct access to internal ERSys state or to resolution engines.

Entity Resolution Engine (ERE)

The Entity Resolution Engine is a supporting application component that provides computational resolution capabilities. It operates under the ERS–ERE asynchronous contract and:

- receives resolution requests from ERS;
- executes resolution algorithms;
- returns candidate resolution results with associated assessment signals.

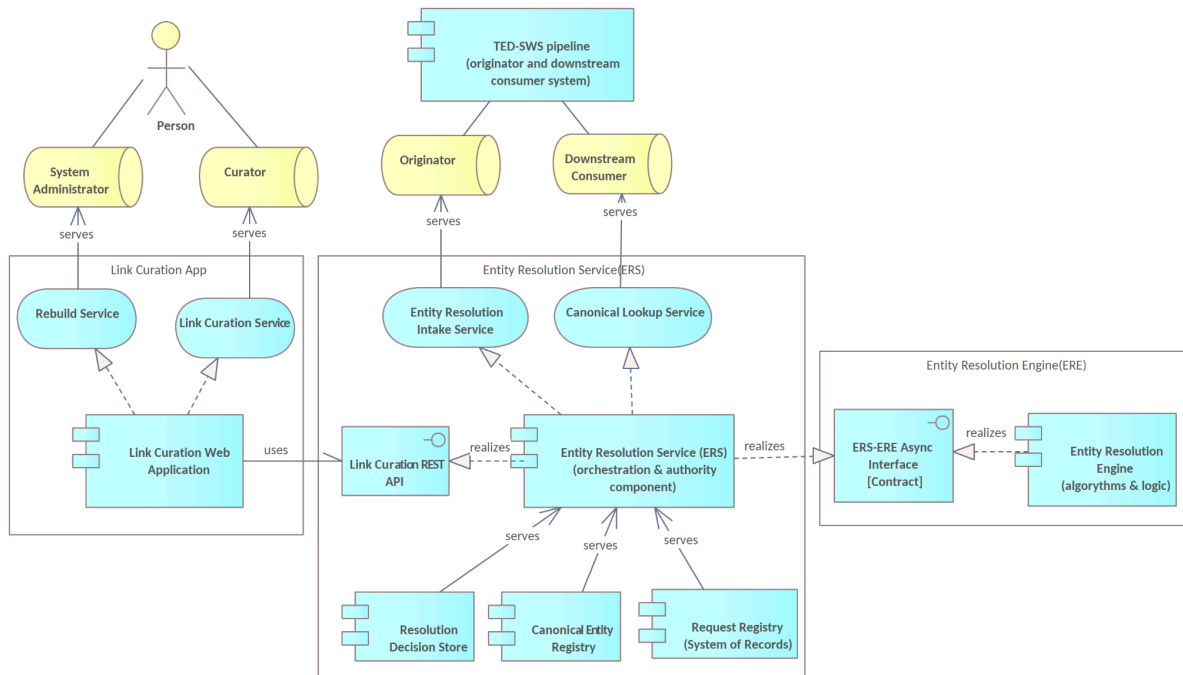
ERE does not issue canonical identifiers and does not persist authoritative state or governance decisions.

Authoritative persistence

Authoritative persistence within ERSys is managed exclusively by ERS and includes:

- the Request Registry, acting as the system of record for submitted entity mentions;
- the Resolution Decision Store;
- the Canonical Entity Registry.

No external application component, including the Link Curation Application and ERE, accesses these stores directly.



This diagram shows the **application-level cooperation structure** within and around ERSys, focusing on responsibility allocation and authority boundaries between application components. The diagram uses **Arch42 Application layer elements**. Application components represent deployable logical units, and application interfaces represent explicit contracts through which all cooperation occurs. Persistence components are shown only to clarify authority over state, not to imply physical storage design.

The central placement of the Entity Resolution Service (ERS) reflects its architectural authority. Supporting components, including the Link Curation Application and the Entity Resolution Engine, are deliberately positioned outside the ERSys authority boundary to emphasise non-authoritative roles.

The diagram does not express runtime order, synchronisation, or lifecycle semantics. All interactions shown are structural and contractual, with behavioural aspects defined exclusively in later use cases and interaction diagrams.

6.3 Interfaces and contracts

All cooperation between application components and external systems is mediated through explicit interfaces and contracts. Business services are realised through one or more application interfaces.

The primary contracts are:

- **ERS Public APIs**, exposing the Entity Resolution Intake Service, Canonical Lookup Service, Link Curation Service, and Rebuild Service to external roles.
- **Link Curation REST API**, used by the Link Curation Application to interact with ERS.
- **ERS-ERE Asynchronous Interface**, defining message structures, correlation identifiers, and delivery expectations for interaction between ERS and ERE.

Detailed interface specifications, such as OpenAPI definitions or message schemas, are maintained separately and referenced from this document. This section establishes interface ownership and architectural role, not syntactic detail.

6.4 Interpretation

The views defined in this section establish the structural foundation for the behavioural descriptions that follow. They define who may interact with ERSys, through which services and interfaces, and which components hold authority over identity and governance.

Subsequent sections describe observable system behaviour through use cases and interaction diagrams. Those descriptions shall not redefine or contradict the structural boundaries and responsibilities established here.

7. Core system capabilities (White Use Cases)

This section defines the core behavioural capabilities of the Entity Resolution System (ERSys) through a set of White Use Cases. White Use Cases are normative. They specify the externally observable behaviour and minimum guarantees that ERSys provides to its actors, independently of internal workflows, algorithms, component structure, or deployment choices.

White Use Cases establish what ERSys does, under which conditions, and with which observable outcomes. They intentionally avoid describing execution order, internal orchestration, component interactions, or technology-specific mechanisms. Such details are refined through Blue Use Cases, Architecture Decision Records (ADRs), and interaction diagrams in later sections.

This section focuses exclusively on the core resolution lifecycle, covering:

- submission and resolution of entity mentions;
- human governance of resolution decisions;
- controlled rebuild of resolution state;
- read-only consultation of aggregated resolution statistics.

Operational and administrative capabilities, such as user management, authentication, configuration of system variants, and operational monitoring, are intentionally excluded from this section and specified elsewhere.

White Use Cases describe behavioural capabilities, while application services, introduced in Section 6, represent stable access points through which those capabilities are exercised. The relationship between use cases and services is not one-to-one. A single use case may involve multiple services, and a single service may support multiple use cases. Service mappings are therefore indicative and non-normative.

The table below summarises the White Use Cases that constitute the core behavioural contract of ERSys and that are specified in detail in the remainder of this section.

White UC ID	Title	Primary Actor	Core Capability	Architectural significance
UC-W1	Resolve Entity Mention	API Client (Originator)	Assign and expose a canonical identifier for an entity mention within bounded time	Establishes the primary client-facing guarantee of ERSys, including non-empty resolution, identifier stability, and bounded response behaviour
UC-W2	Curate Entity Resolution Decisions	Curator	Govern automated resolution outcomes through explicit human decisions	Enforces decision-centric governance, curator authority, and precedence of manual curation over automation
UC-W3	Rebuild Resolution State	System Administrator	Perform controlled, system-level corrective reprocessing for one entity type	Defines explicit corrective authority while preserving canonical identity invariants and curator locks
UC-W4	Consult Resolution Statistics	Curator	Retrieve aggregated, read-only indicators of resolution and curation state	Supports governance oversight and prioritisation without exposing or modifying authoritative state

Each subsection that follows introduces one White Use Case with a brief narrative framing, followed by the full, authoritative use case text.

7.1 Resolve Entity Mention (UC-W1)

Primary Actor: API Client (e.g. TED-SWS, Link Curation Backend, or any authorised originator)

Scope: Entity Resolution System (ERSys)

Stakeholders and Interests

- **API Client:** requires a stable, authoritative canonical identifier for each submitted entity mention, suitable for immediate downstream processing and later reconciliation.
- **Downstream consumers:** depend on canonical identifiers as stable proxies for managed identities, regardless of subsequent resolution or curation activity.
- **ERS operators:** require predictable, bounded API behaviour, strict idempotency, and isolation from engine-level failures.

Brief Description

The API Client submits an Entity Mention to ERSys in order to obtain a canonical identifier representing the managed identity under which the mention is governed.

For each valid Entity Mention, ERSys deterministically derives and returns a canonical identifier according to a standardised derivation rule. The canonical identifier is returned synchronously and within bounded time, independently of the availability or completion of automated resolution processing.

Automated resolution and candidate evaluation may occur asynchronously and may subsequently influence the governance state of the mention through Resolution Decisions, but they do not delay or block the assignment of a canonical identifier.

Goal / Value

Provide a stable and authoritative canonical identifier for each submitted entity mention within bounded time, enabling client systems to perform reliable downstream processing and long-term reconciliation without dependence on resolution engine availability or internal system state.

Trigger

An authorised API Client submits an Entity Mention to the Entity Resolution System in order to obtain or refresh the canonical identifier governing that mention.

Preconditions

- The API Client is authorised to call the resolution API.
- The request includes the required correlation information, including an originator identifier and a request identifier.
- The Entity Mention payload conforms to supported formats and entity types.

Success Guarantee (Postconditions)

For every valid Entity Mention request:

- ERSys registers the request and enforces idempotency based on the originator and request identifiers.
- ERSys assigns a canonical identifier derived deterministically according to the applicable derivation rule.
- ERSys returns the canonical identifier correlated to the submitted request.
- A Resolution Decision exists for the Entity Mention, capturing the current governance state, including any available automated assessment.

Minimal Guarantees (On Failure)

- If the request is syntactically invalid or violates mandatory validation rules, ERSys returns an explicit error response and does not assign a canonical identifier.
- If internal dependencies required for request registration are unavailable, ERSys returns an explicit service error.

- Failures or delays in automated resolution processing do not prevent canonical identifier assignment or response delivery.

Main Success Scenario

1. The API Client submits a resolve request containing a single Entity Mention and the required correlation identifiers.
2. ERSys validates the request and registers it, enforcing idempotency.
3. ERSys deterministically derives the canonical identifier for the Entity Mention.
4. ERSys returns the canonical identifier to the API Client within bounded time.
5. ERSys may initiate or continue asynchronous resolution processing to evaluate candidate alignments and update the associated Resolution Decision.

Extensions

1a. Invalid or malformed request

ERSys rejects the request and returns a validation error describing the issue.

1b. Unsupported entity type or payload format

ERSys rejects the request and returns an explicit error indicating the unsupported type or format.

2a. Repeated request (idempotent replay)

If a request with the same originator and request identifiers is submitted again, ERSys does not create a new identity and returns the canonical identifier previously assigned to the request.

Special Requirements

- The resolve operation is idempotent with respect to request registration and canonical identifier assignment.
- Canonical identifier derivation is deterministic and standardised; the specific derivation mechanism is not exposed via this use case.
- Automated resolution, candidate ranking, and confidence assessment are asynchronous and do not affect the synchronous guarantees of this use case.

Technology and Data Variations

Entity Mention payloads may vary in format and structure, subject to validation and supported entity types.

Frequency of Occurrence

High; expected to be invoked repeatedly as part of automated ingestion and synchronisation pipelines.

Open Issues

None at this level. Detailed error taxonomies and resolution outcome representations are defined in lower-level use cases and interface specifications.

7.2 Curate Resolution Decisions (UC-W2)

Primary Actor: Curator (human user)

Scope: Entity Resolution System (ERSys)

Stakeholders and Interests

- **Curators:** need a clear, authoritative way to review and override automated resolution outcomes in order to ensure correctness and governance compliance.
- **Data governance owners:** require that manual decisions are traceable, authoritative, and consistently applied.
- **Downstream consumers:** depend on stable canonical identifiers that reflect approved governance decisions.
- **ERS operators:** require that manual curation integrates cleanly with automated resolution without breaking system invariants.

Brief Description

The Curator reviews existing Resolution Decisions produced by ERS and performs manual curation actions to govern how Entity Mentions are associated with canonical identifiers.

Manual curation operates exclusively at the level of Resolution Decisions. Curator actions update the governance state of a decision and take precedence over any automated proposals produced by Entity Resolution Engines. ERS projects the outcome of the current Resolution Decision into the Canonical Entity Registry.

Curation does not create new canonical identifiers. It selects among existing canonical identifiers or confirms the default assignment previously derived by ERS.

Goal / Value

Ensure that automated resolution outcomes can be explicitly governed by human curators, so that canonical identity assignments reflect trusted, explainable decisions and remain aligned with domain context and governance requirements.

Trigger

A Curator accesses the Link Curation application to review and act upon one or more Resolution Decisions.

Preconditions

- The Curator is authenticated and authorised to perform curation actions.
- One or more Resolution Decisions exist for review, created via automated resolution or default assignment.

Success Guarantee (Postconditions)

For each curated Resolution Decision:

- The governance state of the Resolution Decision is updated according to the Curator's action.
- Manual curation overrides any automated assessment for that decision.

- The Canonical Entity Registry is updated to reflect the canonical mapping governed by the updated Resolution Decision.
- The updated state is visible to lookup operations and downstream consumers.

Minimal Guarantees (On Failure)

- If a curation action cannot be persisted, the existing Resolution Decision and canonical mapping remain unchanged.
- Partial failures do not leave Resolution Decisions in an inconsistent state.

Main Success Scenario

1. The Curator selects a Resolution Decision for review in the Link Curation application.
2. ERS presents the current decision state and relevant context for that decision.
3. The Curator performs a manual curation action on the Resolution Decision.
4. ERS validates and records the updated governance state of the Resolution Decision.
5. ERS updates the Canonical Entity Registry to reflect the curated canonical mapping.
6. The updated canonical assignment becomes effective for subsequent lookups and downstream processing.

Extensions

2a. Resolution Decision not found or no longer available

ERS reports that the selected decision cannot be curated.

3a. Concurrent modification detected

ERS rejects the curation action and reports a conflict.

Special Requirements

- Manual curation actions are authoritative and dominate automated resolution outcomes.
- Curation affects governance state only and does not modify deterministic identifier derivation rules.
- Curation actions are traceable.

Frequency of Occurrence

Medium; driven by data quality workflows and governance needs.

Notes / Business Rules

- Manual curation establishes the authoritative canonical mapping until explicitly changed by a subsequent curation or rebuild operation.

7.3 Rebuild Resolution State (UC-W3)

Primary Actor: System Administrator

Scope: Entity Resolution System (ERSys)

Stakeholders and Interests

- **System Administrators / ERS Operators:** require a controlled, explicit mechanism to recover from systemic resolution errors, degraded clustering quality, or major configuration changes, while limiting operational risk and ensuring predictable behaviour.
- **Curators:** expect that manually curated decisions remain authoritative and are not silently overridden or lost during corrective operations.
- **Data Consumers:** depend on continuity and long-term stability of canonical identifiers, even when large-scale corrective actions are applied.
- **Data Governance Owners:** require transparency, traceability, and clear responsibility boundaries for disruptive and potentially irreversible corrective actions.
- **ERS Maintainers:** need rebuild semantics that are isolated from normal incremental resolution flows and do not undermine architectural invariants.

Brief Description

This use case describes a system-level corrective capability that allows an authorised administrator to trigger a controlled rebuild of the resolution state for a single selected entity type.

A rebuild is an explicit, exceptional operation that reprocesses all known Entity Mentions of the selected entity type by generating new automated resolution proposals and integrating them through the normal decision-centric governance model. Rebuild operations do not change canonical identifier derivation rules and do not bypass Resolution Decisions. Instead, they regenerate decision context at scale, while preserving authoritative manual curation constraints.

Rebuild operations are intentionally scoped to one entity type at a time to limit operational blast radius, preserve the stability of unaffected canonical domains, and align corrective action with the specific quality, configuration, or governance concerns that motivated the rebuild. Rebuilds are disruptive and long-running, but are designed to be transparent, traceable, and operationally isolated from normal incremental resolution flows.

Goal / Value

Restore or improve the global coherence and internal consistency of resolution outcomes for a selected entity type by regenerating and re-integrating automated proposals under a controlled rebuild operation, while preserving governed knowledge, deterministic identity rules, and downstream system stability.

Scope (System-of-Interest)

The Entity Resolution System (ERSys), including:

- regeneration of automated resolution proposals for the selected entity type,
- integration of rebuild outcomes via Resolution Decisions,
- maintenance of the Canonical Entity Registry as a projection of the currently governed state.

Out of Scope

- Re-resolution of a single Entity Mention via normal APIs.
- Localised corrective actions limited to a single canonical cluster.
- External, client-triggered rebuild requests.
- Administrative tooling outside the Link Curation application (e.g. ad-hoc scripts or CLI-based recovery).

Trigger

An authorised System Administrator explicitly initiates a rebuild operation for a specific entity type, typically in response to detected systemic resolution issues, degraded clustering quality, or significant configuration changes.

Preconditions

- The System Administrator is authenticated and authorised to initiate rebuild operations.
- No other rebuild operation is currently active.
- The selected entity type is in a stable operational state suitable for rebuild initiation.

Success Guarantees (Postconditions)

For the selected entity type:

- All known Entity Mentions remain associated with exactly one canonical identifier, in accordance with deterministic derivation rules.
- Existing manual curation constraints remain authoritative and are preserved throughout the rebuild.
- Resolution Decisions governing mention-to-canonical mappings may be superseded by rebuild outcomes, except where constrained by manual curation.
- The Canonical Entity Registry reflects the currently governed state resulting from rebuild integration.
- The entity type returns to normal incremental resolution mode after rebuild completion or accepted partial completion.

Minimal Guarantees (On Failure)

- All successfully integrated Resolution Decisions applied prior to failure remain effective.
- No partial or inconsistent canonical state is exposed.
- Canonical identifier derivation rules remain unchanged.
- The entity type is returned to normal incremental resolution mode.
- The administrator may re-initiate the rebuild after corrective action.

Main Success Scenario

1. The System Administrator initiates a rebuild operation for a selected entity type.
2. ERSys reprocesses resolution state for that entity type under the decision-centric governance model and converges to a consistent, governed canonical state that preserves identity invariants and manual curation constraints.

Extensions

2a. Rebuild cannot be initiated

If preconditions for rebuild initiation are not met, ERSys rejects the request and no resolution state is modified.

2b. Rebuild completes partially

If rebuild processing is interrupted, all successfully integrated outcomes remain effective and the entity type remains in a consistent, governed state.

Special Requirements

- Rebuild operations shall not alter canonical identifier derivation rules.
- Rebuild operations shall not bypass or invalidate authoritative manual curation constraints.
- Rebuild operations shall preserve the invariant that each Entity Mention is governed by exactly one canonical identifier.

Notes / Business Rules

- At most one rebuild operation may be active at any time, and it applies to exactly one entity type.
- During a rebuild, normal resolution and curation operations remain available; their outcomes may be superseded by rebuild-generated decisions for the affected entity type.
- Rebuild operations operate exclusively through the Resolution Decision model and do not directly mutate canonical state outside of governed projections.
- Rebuild behaviour (e.g. state-aware versus destructive regeneration of automated proposals) is a system configuration decision; state-aware rebuild is recommended for production environments.

Frequency of Occurrence

Low; rebuild operations are exceptional and expected to be invoked rarely.

Open Issues

- Criteria for determining rebuild completion versus accepted partial completion.
- Operator guidance for coordinating rebuilds with ongoing curation activity.

UC-W4: Consult Resolution Statistics

Primary Actor: Curator (human user)

Scope: Entity Resolution System (ERSys)

Stakeholders and Interests

- **Curators:** Want a quick, reliable overview of the current resolution and curation workload in order to prioritise review efforts and understand progress.

- **System Administrators / ERS Operators:** Need lightweight observability to assess system behaviour, backlog evolution, and the balance between automated and manual decisions.
- **Data Governance Owners:** Require high-level indicators demonstrating that governance processes are active, proportionate, and effective.
- **Data Consumers:** Indirectly depend on stable resolution quality, which is reflected in aggregate statistics rather than individual decisions.

Brief Description

This use case describes how a human curator consults aggregated, read-only statistics about the current state of entity resolution and curation through the Link Curation application. The system provides high-level indicators, scoped by entity type and optionally by time window, such as the volume of pending and resolved decisions, the proportion of automated versus manually curated outcomes, and the overall size of the canonical entity set.

The use case is intentionally informational and non-intrusive. It supports situational awareness, prioritisation, and governance oversight without exposing individual entities or decisions, and without allowing any modification of resolution state.

Goal / Value

Provide a simple, aggregated view of resolution and curation statistics so that human operators can understand the current state of the Entity Resolution System without inspecting individual decisions or entities.

Scope (System-of-Interest)

The Link Curation application and the Entity Resolution Service (ERS), limited to the retrieval and presentation of aggregated, read-only statistics derived from existing operational stores.

Implementation note (non-normative): statistics are derived from the Decision Store and the Canonical Entity Registry.

Out of Scope

- Inspection or modification of individual decisions or entities.
- Operational monitoring, alerting, or performance metrics.
- Historical trend analysis or time-series dashboards.
- Administrative or corrective actions.

Trigger

A curator explicitly requests to view resolution statistics, typically to assess current workload, progress, or overall resolution health.

Preconditions

- The curator is authenticated and authorised to access statistical views.
- The ERS statistics endpoint is available.

Success Guarantees (Postconditions)

- The user can retrieve aggregated statistics scoped by entity type and, where applicable, by time frame.
- Statistics reflect the current observable state of the system at the time of the request.
- The operation is strictly read-only and does not alter any system state.

Minimal Guarantees (On Failure)

- No system state is modified.
- The user is informed that statistics are temporarily unavailable.

Main Success Scenario

1. The Curator requests aggregated resolution statistics.
2. ERSys returns a read-only summary reflecting the current observable resolution and curation state, without modifying any authoritative records.

Extensions

1a. Statistics unavailable

If statistics cannot be retrieved, ERSys reports temporary unavailability and no system state is modified.

Special Requirements

- Statistical views shall be strictly read-only.
- Statistical views shall not expose individual entities, decisions, or canonical identifiers.
- Statistical views shall not be interpreted as audit or compliance evidence.

Notes / Business Rules

- Statistics are derived from existing operational stores and are not independently curated or corrected.
- Counts are indicative and may change as resolution and curation activities continue.
- Statistics may be temporarily inconsistent during long-running rebuild operations.
- Statistics are indicative operational signals and must not be interpreted as audit evidence or compliance artefacts.

7.5 Relationship to detailed behaviour and architectural decisions

The White Use Cases defined in this section constitute the normative behavioural contract of the Entity Resolution System. They define the externally observable capabilities and guarantees that ERSys must uphold.

Detailed behaviour is refined through Blue Use Cases, Architecture Decision Records (ADRs), and interaction diagrams. These artefacts elaborate internal responsibilities, constraints, and interaction patterns. They must conform to, and must not weaken or contradict, the White Use Cases defined here.

In case of ambiguity between behavioural descriptions, precedence is governed by Section 0.2. White Use Cases take precedence over explanatory architecture text and implementation artefacts. Architectural invariants defined in Section 4 further constrain the interpretation and implementation of the behaviours described in this section.